



Sorting

CEN207 Data Structures — Week 10

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	Cost model · bubble, selection, insertion, shell sort
2	Merge sort (top-down, bottom-up) · quick sort (Lomuto, Hoare, worst case)
3	Counting, radix, bucket sort · stability · empirical comparison · choosing

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
Bubble, selection, insertion, shell sort	Sections 2–5
Merge sort (top-down / bottom-up)	Section 6
Quick sort (Lomuto / Hoare / worst case)	Section 7
Counting, radix, bucket sort	Sections 8–10
Stability, comparison, choosing	Sections 11–13

How the code examples work

- Every algorithm has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-10/c/` and `code/week-10/java/`
- Each program's expected output is in the week notes

Recap — arrays and Big-O (Week 1)

- Every sort today works on a plain array, $O(1)$ access
- Best / average / worst case is not decoration
- It is *why* eleven different sorts exist, not one
- Watch the same input give wildly different costs

Recap — heap sort (Week 4)

- You already built an $O(n \log n)$, in-place sort
- Keep it as today's yardstick throughout
- Merge sort matches its guarantee, at $O(n)$ extra cost
- Quick sort matches it only on **average**

Two genuinely new ideas

- **Divide and conquer**: split, solve each half, combine
- Powers merge sort and quick sort (Sections 6–7)
- **Non-comparison sorting**: never compare two keys
- Powers counting, radix, bucket sort (Sections 8–10)

Map of the week — at a glance

$O(n^2)$ family	$O(n \log n)$ guaranteed
Bubble, selection, insertion, shell	Merge sort (top-down, bottom-up)
Average $O(n \log n)$	Non-comparison, $O(n+k)$
Quick sort (Lomuto, Hoare)	Counting, radix, bucket sort

Mini-quiz

Every sort you have met before today assumes one thing about how order is decided. What is it, and which three sorts today break that rule?

Answer

Comparing two keys with `<` or `>`.

Counting, radix, and bucket sort (Sections 8–10)
never compare keys — they compute locations directly.

1. The Sorting Problem, and How to Measure It

A question to start

Sorting has been solved a dozen different ways in the history of computing. Given how simple the problem sounds, why does it need so many answers?

The sorting problem, restated

- Array of `n` values, a rule ("is `a` less than `b` ?")
- Rearrange so every element is `<=` the next one
- Freedom: how much extra memory, what you exploit
- Freedom: whether equal elements may change order

Measuring cost: two numbers

Quantity	Counts	Matters when
comparisons	every <code><</code> / <code>></code> / <code>==</code>	the classic cost measure
moves / writes / swaps	every array write	records are big, or writes are expensive

Stability, defined

- Equal keys keep their **input order** in the output
- Not about correctness — the output is still sorted
- An **extra** guarantee some algorithms give for free
- Gets a dedicated animation in Section 11

In place, or extra memory?

- **In place:** rearranges with $O(1)$ extra memory
- Bubble, selection, insertion, shell, quick sort: in place
- Merge sort needs $O(n)$ extra — the price of its guarantee
- Counting/radix sort need $O(n+k)$ extra too

2. Bubble Sort

A question to start

Comparing two neighbors and swapping them if they are backwards is the most natural sorting instinct there is. How far does that idea alone get you?

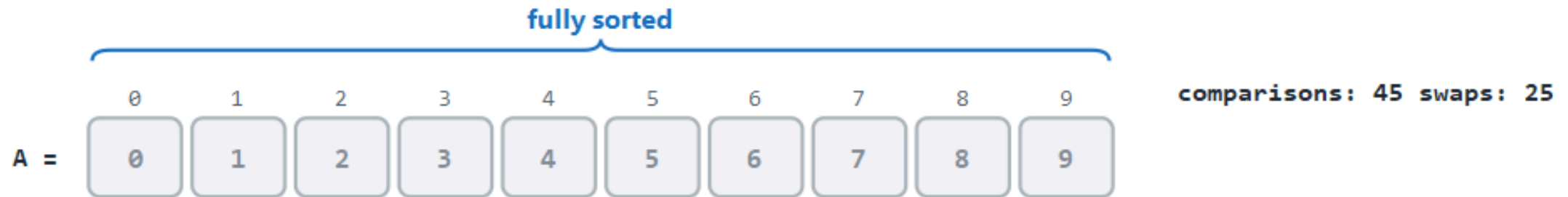
A short history

- No single named inventor, unlike quick or merge sort
- One of the oldest sorting ideas in computing
- Appears in the literature as early as the mid-1950s
- Taught first because it needs no cleverness to invent

The bubble sort idea

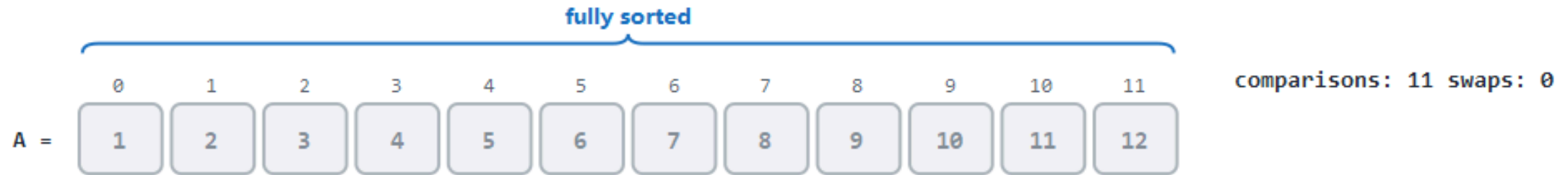
- Walk left to right, compare each neighbor pair
- Swap if out of order — the larger value "bubbles" right
- Repeat the whole walk, one cell shorter each time
- **Early exit:** a pass with zero swaps means done

Bubble sort, step by step



Done: $[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$. Total 45 comparisons, 25 swaps. Worst case $O(n^2)$; thanks to the early exit, best case (already sorted) is $O(n)$.

Edge case — already sorted



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Total 11 comparisons, 0 swaps. Worst case $O(n^2)$; thanks to the early exit, best case (already sorted) is $O(n)$.

Code — the pass and the early exit

```
for (int pass = 0; pass < n - 1; pass++) {  
    int swapped = 0;  
    for (int i = 0; i < n - 1 - pass; i++) {  
        if (a[i] > a[i + 1]) {  
            int tmp = a[i]; a[i] = a[i + 1];  
            a[i + 1] = tmp; swapped = 1;  
        }  
    }  
    if (!swapped) break;  
}
```

Complexity and mistakes

- Best case $O(n)$: early exit on already-sorted input
- Worst/average case $O(n^2)$: reverse-sorted or random
- Common bug: using `>=`, which breaks stability
- Common bug: forgetting the early-exit flag entirely

Mini-quiz

Why is bubble sort **stable** — what specific line of code guarantees that two equal elements are never swapped past each other?

Answer

The strict `>` in `if (a[i] > a[i+1])`.

Equal elements never satisfy `>`, so they are never swapped — their input order survives.

3. Selection Sort

A question to start

Bubble sort's reverse-sorted case did 66 swaps to sort 12 values. What if you scanned once for the true minimum, and moved it directly into place?

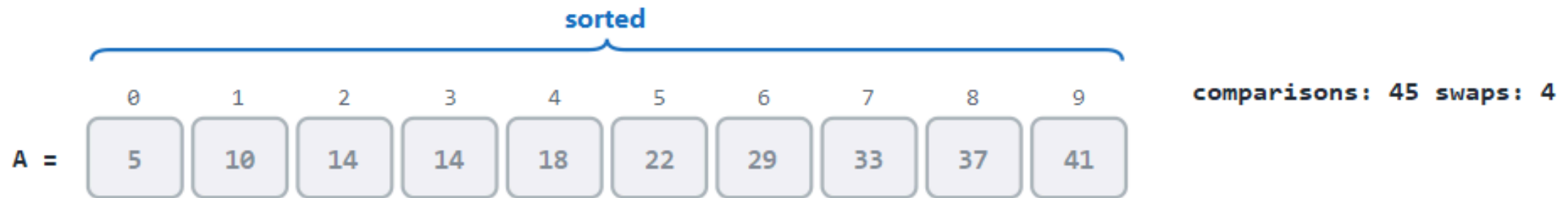
A short history

- Like bubble sort, no single named inventor
- The direct algorithmic form of "pick the smallest left"
- At least as old as the idea of sorting itself
- Valued today for its minimal-write guarantee

The selection sort idea

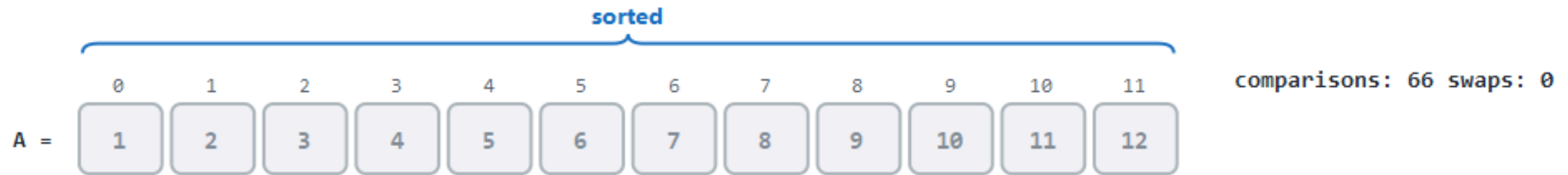
- For each `i`, scan `i+1..n-1` for the minimum's index
- Swap it into position `i` — skip if already there
- Sorted region grows from the **left** (bubble: right)
- At most `n-1` swaps, ever — but always $O(n^2)$ scans

Selection sort, step by step



Done: [5, 10, 14, 14, 18, 22, 29, 33, 37, 41]. Total 45 comparisons, only 4 swaps. It always does $O(n^2)$ comparisons (no early exit) but at most $n-1$ swaps -- an advantage when writes are expensive.

Edge case — already sorted



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Total 66 comparisons, only 0 swaps. It always does $O(n^2)$ comparisons (no early exit) but at most $n-1$ swaps -- an advantage when writes are expensive.

Code — find the minimum, swap once

```
for (int i = 0; i < n - 1; i++) {  
    int min_idx = i;  
    for (int j = i + 1; j < n; j++)  
        if (a[j] < a[min_idx]) min_idx = j;  
    if (min_idx != i) {  
        int tmp = a[i];  
        a[i] = a[min_idx]; a[min_idx] = tmp;  
    }  
}
```

Complexity and mistakes

- Best/average/worst: all $O(n^2)$ comparisons, always
- At most $n-1$ swaps — the fewest of any sort today
- Common bug: swapping unconditionally, wasting writes
- **Not stable**: the long-range swap can jump a tie

Mini-quiz

Selection sort does at most $n-1$ swaps, no matter the input. Why does that bound hold regardless of how scrambled the array is?

Answer

At most one swap per outer-loop step.

Each element that needs to move, moves once,
directly to its final spot — $n-1$ steps, $n-1$ swaps max.

4. Insertion Sort

A question to start

Think about sorting a hand of playing cards.
You never compare every card to every other
card. What do you actually do instead?

A short history

- The card-sorting idea itself — no single inventor
- One of the oldest, most intuitive sorting methods
- The instructor's own board drawings use exactly this model
- Real library sorts fall back to it for small sub-arrays

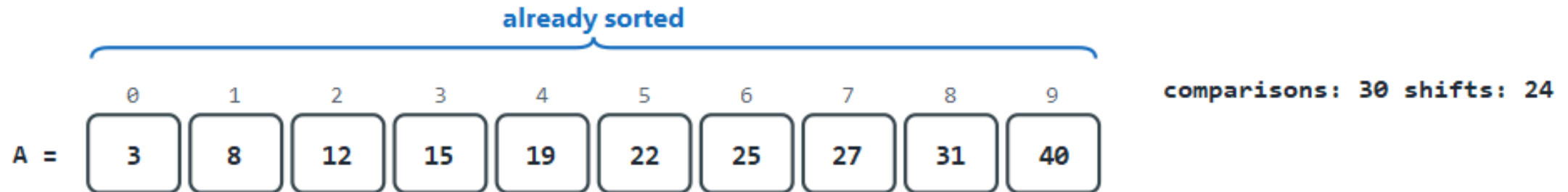
The insertion sort idea

- $A[0..i-1]$ is always **already sorted**
- Pull $A[i]$ out as the **key** (the red box)
- Shift every $> \text{key}$ element one cell right
- Drop the key into the hole it opened

The board-drawing model

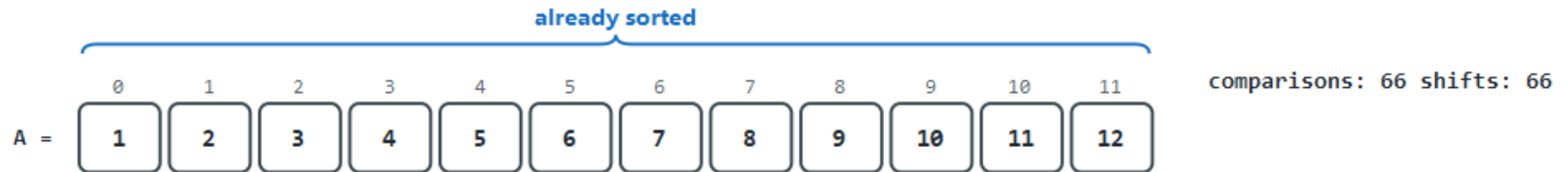
- **Brace** over `A[0..i-1]` : "already sorted"
- **Red box**: the key, pulled out of the array
- `<= key` region stays; `> key` region shifts
- **Shift arrow**: the hole moving one step left

Insertion sort, step by step



Done: [3, 8, 12, 15, 19, 22, 25, 27, 31, 40]. Total 30 comparisons, 24 shifts. Best case (already sorted) $O(n)$; worst case (reverse sorted) $O(n^2)$. It is stable.

Edge case — reverse sorted



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Total 66 comparisons, 66 shifts. Best case (already sorted) $O(n)$; worst case (reverse sorted) $O(n^2)$. It is stable.

Code — pull the key, shift, drop it in

```
for (int i = 1; i < n; i++) {  
    int key = a[i];  
    int j = i - 1;  
    while (j >= 0 && a[j] > key) {  
        a[j + 1] = a[j];  
        j--;  
    }  
    a[j + 1] = key;  
}
```

Complexity and mistakes

- Best case $O(n)$: already sorted, one check per `i`
- Worst case $O(n^2)$: reverse sorted, full shifts
- Common bug: `>=` instead of `>` breaks stability
- Common bug: checking `a[j] > key` before `j >= 0`

Mini-quiz

Why is insertion sort's best case exactly $O(n)$, and what specific kind of real-world data makes that best case the common case?

Answer

Already-sorted (or nearly so) input.

Each `a[j] <= key` check succeeds immediately, so most of the n outer steps cost only $O(1)$.



5. Shell Sort

A question to start

Insertion sort is slow exactly when an element starts far from home. What if you closed most of that distance first, in big jumps?

A short history

- Donald L. Shell, "A High-Speed Sorting Procedure"
- *Communications of the ACM*, 1959
- The first sorting algorithm to beat plain $O(n^2)$
- Gap-sequence choice remains an open research area

The shell sort idea

- Insertion sort, but comparing `gap` apart, not 1 apart
- Start `gap = n/2`, halve every round, end at `gap = 1`
- Large gaps move far-out elements home in one jump
- By `gap = 1`, the array is "almost sorted" — cheap pass

Shell sort, step by step



gap = 1

comparisons: 31 shifts: 14

Done: [2, 5, 9, 12, 17, 23, 28, 33, 41, 46]. Total 31 comparisons, 14 shifts. Worst case $O(n^2)$ with this gap sequence, but far faster than plain insertion sort in practice.

Edge case — reverse sorted



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Total 39 comparisons, 24 shifts. Worst case $O(n^2)$ with this gap sequence, but far faster than plain insertion sort in practice.

Code — insertion sort, gapped

```
for (int gap = n/2; gap > 0; gap /= 2) {  
    for (int i = gap; i < n; i++) {  
        int key = a[i];  
        int j = i;  
        while (j >= gap && a[j-gap] > key) {  
            a[j] = a[j - gap];  
            j -= gap;  
        }  
        a[j] = key;  
    }  
}
```

Complexity and mistakes

- Worst case $O(n^2)$ with this simple gap sequence
- Far faster than plain insertion sort in practice
- Better sequences (Hibbard, Sedgewick) lower the bound
- **Not stable**: gapped moves can cross equal elements

Mini-quiz

In the reverse-sorted example, the smallest value starts at the last index. How many total shifts does it take across all three gap rounds?

Answer

Four shifts total (sizes 6, 3, 1, 1) — covering the same 11-position distance plain insertion sort would need 11 single-cell shifts to cover.

6. Merge Sort

A question to start

What if, instead of moving elements around inside one array, you split it in half, sorted each half completely, then just combined them?

A short history

- John von Neumann, 1945
- One of the first algorithms for a stored-program computer
- Founding example of **divide and conquer** in computing
- The pattern reappears constantly for the rest of this course

The divide-and-conquer idea

- Split $[lo, hi)$ at its midpoint into two halves
- Recursively sort each half the same way
- **Merge** the two sorted halves with a helper array
- $\log_2(n)$ levels $\times$ $O(n)$ per level = $O(n \log n)$

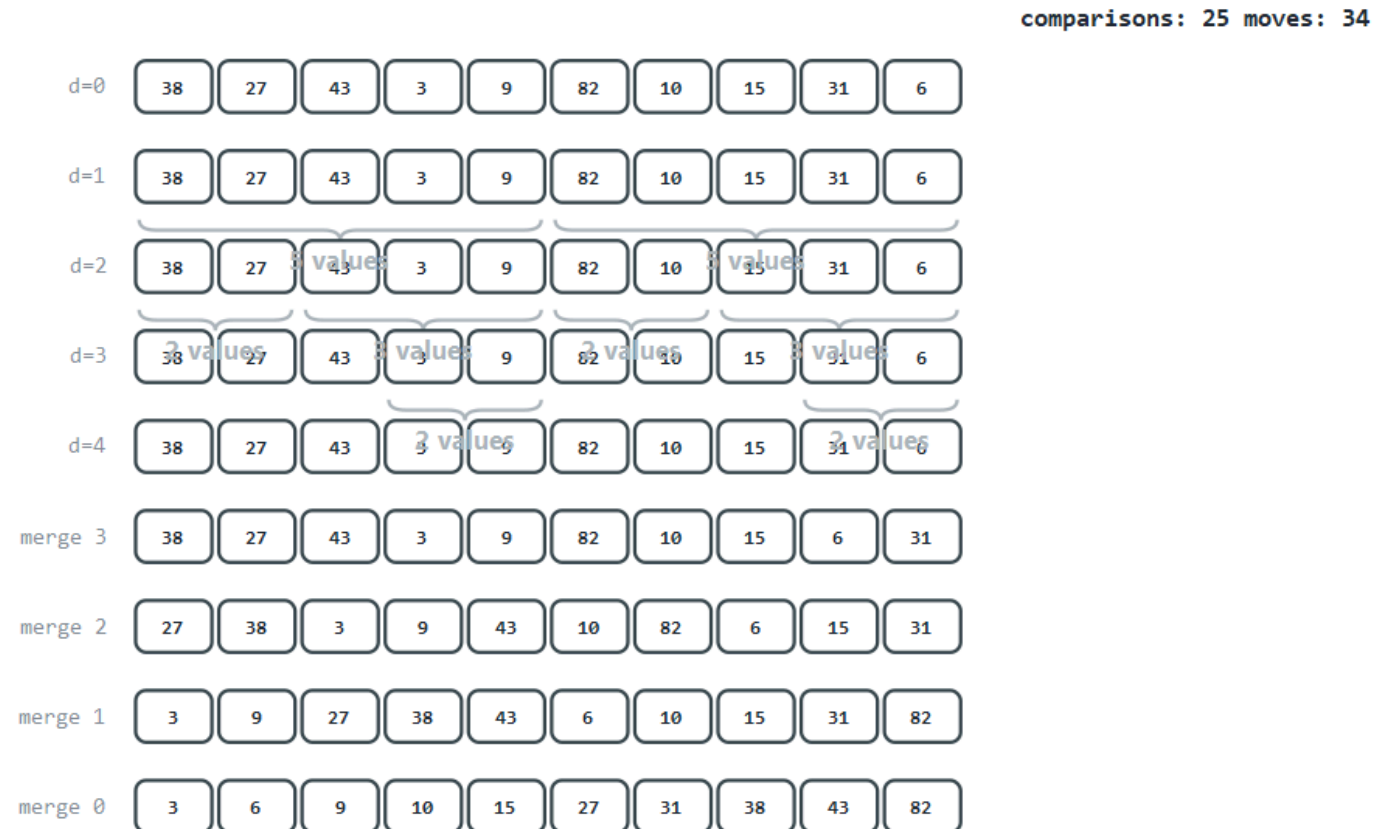
Why the guarantee never breaks

- No early exit, no data-dependent shortcuts anywhere
- Every level always touches every element once
- Price: an auxiliary array of size `n` — $O(n)$ extra
- Two equivalent forms: top-down, bottom-up

Top-down — recursion depth as rows

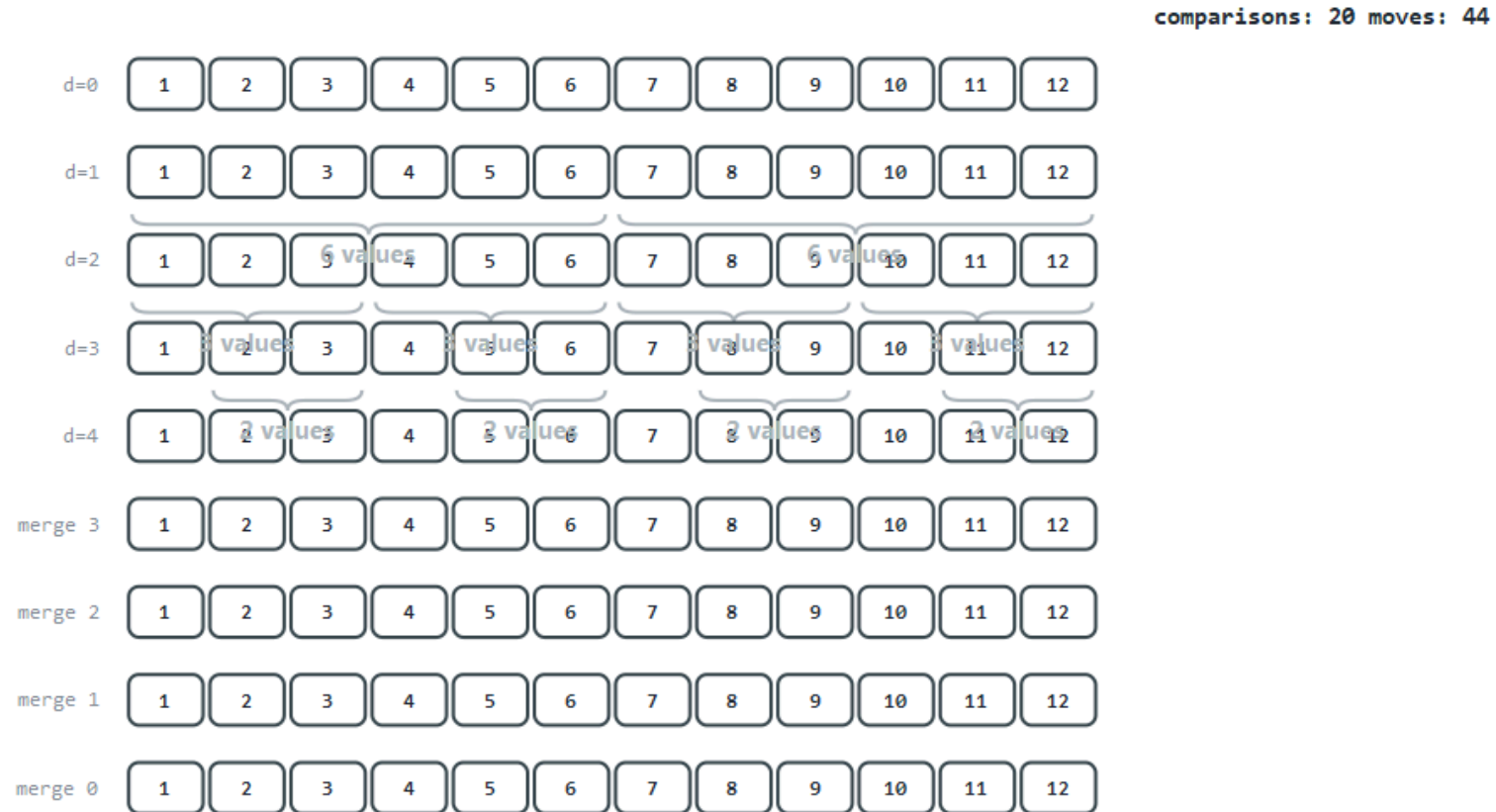
- $d=0$: the whole array; $d=1$: two halves; ...
- Splitting never changes values, only brace boundaries
- Then merging goes back up the same rows
- Last row: the fully sorted array

Merge sort (top-down), step by step



Done: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Total 25 comparisons, 34 moves. Always $O(n \log n)$; uses $O(n)$ extra memory for tmp, but it is stable.

Edge case — already sorted



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Total 20 comparisons, 44 moves. Always $O(n \log n)$; uses $O(n)$ extra memory for tmp, but it is stable.

Code — the merge step

```
int i = lo, j = mid, k = lo;
while (i < mid && j < hi)
    tmp[k++] = (a[i] <= a[j])
                ? a[i++] : a[j++];
while (i < mid) tmp[k++] = a[i++];
while (j < hi)  tmp[k++] = a[j++];
for (int x=lo; x<hi; x++) a[x]=tmp[x];
```

Code — the recursive driver

```
void merge_sort(int a[], int lo,
                int hi, int tmp[]) {
    if (hi - lo <= 1) return;
    int mid = lo + (hi - lo) / 2;
    merge_sort(a, lo, mid, tmp);
    merge_sort(a, mid, hi, tmp);
    merge(a, lo, mid, hi, tmp);
}
```

Bottom-up — no recursion at all

- Every element starts as a sorted run of width 1
- Merge adjacent runs into width-2, then width-4, ...
- Doubling stops once one run covers the whole array
- Same `merge()`, reached by a loop instead of a call stack

Merge sort (bottom-up), step by step

comparisons: 25 moves: 36



Done: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Total 25 comparisons, 36 moves. Same $O(n \log n)$ cost, but no recursion call stack is ever used.

Edge case — n a power of two

comparisons: 47 moves: 64



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]. Total 47 comparisons, 64 moves. Same $O(n \log n)$ cost, but no recursion call stack is ever used.

Code — the width-doubling loop

```
for (int width=1; width<n; width*=2) {  
    for (int lo=0; lo<n-width;  
        lo += 2*width) {  
        int mid = lo + width;  
        int hi = min(mid+width, n);  
        merge(a, lo, mid, hi, tmp);  
    }  
}
```

Complexity and mistakes

- $O(n \log n)$ in every case — best, average, worst
- $O(n)$ extra memory — the price of that guarantee
- **Stable**: the `<=` tie-break always prefers the left run
- Common bug: `<` instead of `<=` breaks stability silently

Mini-quiz

Both top-down and bottom-up merge sort do the same total work. Name one real advantage bottom-up has, and one top-down has in return.

Answer

Bottom-up: never uses the call stack, cannot overflow it. **Top-down:** its recursive structure is easier to adapt — e.g. switch to insertion sort early.

7. Quick Sort

A question to start

Merge sort's $O(n \log n)$ costs $O(n)$ extra memory.
Is there a divide-and-conquer sort that is just
as fast on average, but sorts in place?

A short history

- **C. A. R. Hoare**, 1959–1960, Moscow State University
- Published as "Quicksort", *The Computer Journal*, 1962
- Still the default array sort in many language libraries
- One of the most consequential algorithms in this course

The quick sort idea

- Pick one element of the range as the **pivot**
- **Partition:** `<= pivot` left, `>= pivot` right
- Recursively quick-sort left part and right part
- No separate "combine" step — partitioning does the work

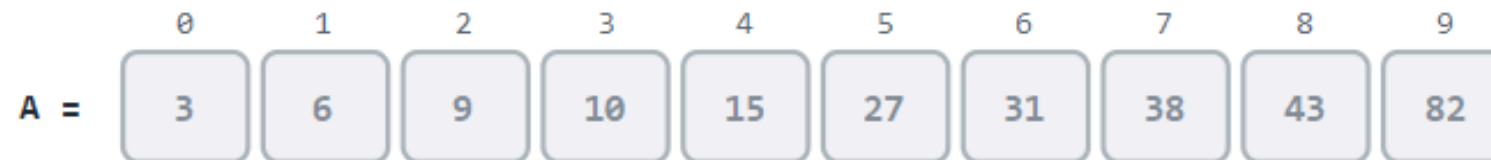
Two classic partition schemes

- **Lomuto**: pivot = last element, one forward scan
- **Hoare**: pivot = first element, two inward scans
- Both studied side by side — they differ in a way
- that trips up almost everyone the first time

Lomuto partition — the idea

- Pivot = the range's **last** element
- **i** marks the boundary of the " **$\leq$ pivot**" region
- **j** scans left to right; **$a[j] \leq \text{pivot}$** advances **i**
- Pivot swaps to **i+1** — its final, correct position

Quick sort (Lomuto), step by step

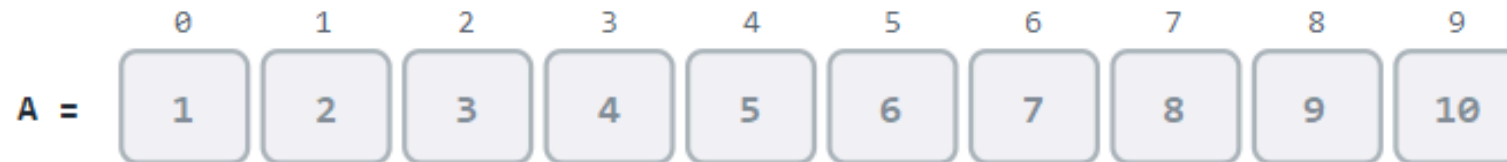


comparisons: 25 swaps: 11

Done: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Total 25 comparisons, 11 swaps. Average $O(n \log n)$; already-sorted/reverse-sorted input gives $O(n^2)$ (see quick-sort-worst-case).

Edge case — already sorted

comparisons: 45 swaps: 9



Done: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Total 45 comparisons, 9 swaps. Average $O(n \log n)$; already-sorted/reverse-sorted input gives $O(n^2)$ (see quick-sort-worst-case).

Code — Lomuto partition

```
int pivot = a[hi];
int i = lo - 1;
for (int j = lo; j < hi; j++) {
    if (a[j] <= pivot) {
        i++;
        int t=a[i]; a[i]=a[j]; a[j]=t;
    }
}
int t=a[i+1]; a[i+1]=a[hi]; a[hi]=t;
return i + 1;
```

Code — Lomuto's recursion

```
void quick_sort_lomuto(  
    int a[], int lo, int hi) {  
    if (lo < hi) {  
        int p = partition_lomuto(  
            a, lo, hi);  
        quick_sort_lomuto(a, lo, p-1);  
        quick_sort_lomuto(a, p+1, hi);  
    }  
}
```

Hoare partition — the idea

- Pivot = the range's **first** element
- Two pointers `i`, `j` scan **inward** from both ends
- Swap when a pair is found out of place; repeat
- Pivot's final resting spot is **not** guaranteed to be `j`

Quick sort (Hoare), step by step

	0	1	2	3	4	5	6	7	8	9
A =	3	6	9	10	15	27	31	38	43	82

comparisons: 55 swaps: 12

Done: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Total 55 comparisons, 12 swaps. Hoare typically does fewer swaps than Lomuto; average $O(n \log n)$.

Edge case — all equal values



comparisons: 48 swaps: 15

Done: [5, 5, 5, 5, 5, 5, 5, 5, 5, 5]. Total 48 comparisons, 15 swaps. Hoare typically does fewer swaps than Lomuto; average $O(n \log n)$.

Code — Hoare partition

```
int pivot = a[lo];
int i = lo - 1, j = hi + 1;
while (1) {
    do { i++; } while (a[i] < pivot);
    do { j--; } while (a[j] > pivot);
    if (i >= j) return j;
    int t=a[i]; a[i]=a[j]; a[j]=t;
}
```

Code — Hoare's recursion trap

```
void quick_sort_hoare(  
    int a[], int lo, int hi) {  
    if (lo < hi) {  
        int p = partition_hoare(  
            a, lo, hi);  
        quick_sort_hoare(a, lo, p);  
        quick_sort_hoare(a, p+1, hi);  
    }  
}
```

Quick sort's worst case

- Both schemes above pick a **fixed corner** as pivot
- Always the smallest/largest element = worst split
- Sorted or reverse-sorted input triggers it directly
- **Median-of-three** is the standard real-world defense

Comparing pivot strategies

same input, different pivot choice

first element



comparisons: 45 calls: 9

middle index



comparisons: 19 calls: 6

median-of-three



comparisons: 19 calls: 6

Done. first element: 45 comparisons, depth 9; middle index: 19 comparisons, depth 3; median-of-three: 19 comparisons, depth 3. Always picking a fixed corner element (first or last) as the pivot degrades to $O(n^2)$ on already-sorted/reverse-sorted input; the middle index or median-of-three stay balanced on this input.

Edge case — random input

same input, different pivot choice

first element



comparisons: 26 calls: 7

middle index



comparisons: 27 calls: 7

median-of-three



comparisons: 21 calls: 6

Done. first element: 26 comparisons, depth 6; middle index: 27 comparisons, depth 6; median-of-three: 21 comparisons, depth 4. Always picking a fixed corner element (first or last) as the pivot degrades to $O(n^2)$ on already-sorted/reverse-sorted input; the middle index or median-of-three stay balanced on this input.

The numbers, made concrete

Strategy	Comparisons (n=14, sorted)	Depth
First element	91	13
Median-of-three	31	3

The comparison-sorting lower bound

- $n!$ possible orderings of n distinct elements
- A decision tree distinguishing all of them needs
- at least $\log_2(n!)$ levels — which is $O(n \log n)$
- No comparison-based sort can ever beat this bound

Complexity and mistakes

- **Average $O(n \log n)$** , in place — the fastest in practice
- **Worst $O(n^2)$** on a badly-chosen, adversarial pivot
- Common bug: mixing Lomuto's `p-1` with Hoare's `p`
- **Not stable**: both schemes swap across long distances

Mini-quiz

Explain in one sentence why Hoare's partition must recurse on (lo, p) and $(p+1, hi)$, not $(lo, p-1)$ like Lomuto's.

Answer

Hoare never guarantees the pivot sits at p — only that everything $\leq p$ is $\leq \text{pivot}$. Excluding p (as $p-1$ would) could drop an unsorted element.

8. Counting Sort

A question to start

The $O(n \log n)$ lower bound assumes you only know how two keys compare. What if you know every value is a small, non-negative integer?

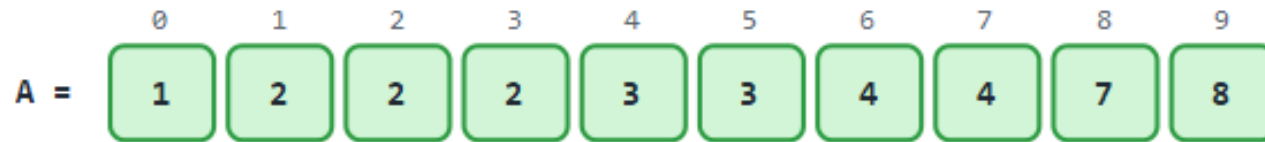
A short history

- **Harold H. Seward**, MIT master's thesis, 1954
- The simplest way to beat the comparison lower bound
- Requires knowing the key range `0..maxVal` in advance
- Foundation for radix sort (Section 9)

The counting sort idea

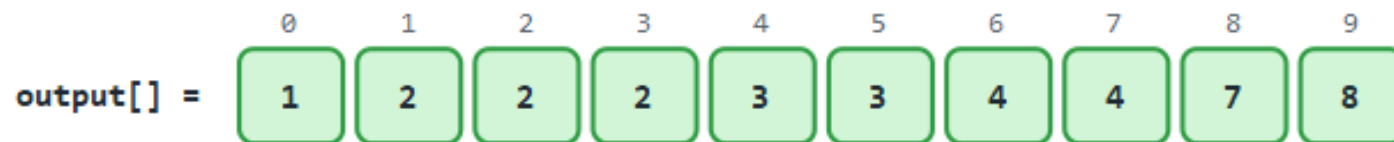
- Count occurrences of each value into `count[]`
- Turn counts **cumulative**: `count[v]` = "how many `<= v`"
- Place each value at `output[count[a[i]]-1]`, decrement
- Scan **backwards** — this is what keeps it stable

Counting sort, step by step



comparisons: 0 (non-comparison sort)

writes: 20



Done: [1, 2, 2, 2, 3, 3, 4, 4, 7, 8]. Total 20 writes, 0 comparisons. $O(n + k)$ time and $O(n + k)$ extra memory ($k = \text{maxVal}$). It is stable. Wasteful when k is much larger than n (see "sparse range").

Edge case — sparse range

A =

0	1	2	3	4	5	6	7	8	9
0	1	3	6	7	8	9	12	14	15

comparisons: 0 (non-comparison sort)
writes: 20

count[] =

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	2	3	3	3	4	5	6	7	7	7	8	8	9

output[] =

0	1	2	3	4	5	6	7	8	9
0	1	3	6	7	8	9	12	14	15

Done: [0, 1, 3, 6, 7, 8, 9, 12, 14, 15]. Total 20 writes, 0 comparisons. $O(n + k)$ time and $O(n + k)$ extra memory ($k = \text{maxVal}$). It is stable. Wasteful when k is much larger than n (see "sparse range").

Code — count, then accumulate

```
int count[max_val + 1] = {0};  
for (int i = 0; i < n; i++)  
    count[a[i]]++;  
for (int v = 1; v <= max_val; v++)  
    count[v] += count[v - 1];
```

Code — place, scanning backwards

```
for (int i = n - 1; i >= 0; i--) {  
    output[count[a[i]] - 1] = a[i];  
    count[a[i]]--;  
}  
for (int i = 0; i < n; i++)  
    a[i] = output[i];
```

Complexity and mistakes

- $O(n + k)$ time and space, $k = \text{maxVal}$ — every case
- Beats $O(n \log n)$ genuinely, when k is not $\gg n$
- Wasteful when k is much larger than n (sparse example)
- **Stable** by construction, via the backward scan

Mini-quiz

Two equal values $a[p]$ and $a[q]$, with $p < q$.
Trace the backward scan — why does $a[p]$
always end up to the LEFT of $a[q]$ in the output?

Answer

`a[q]` is placed first (scan goes $n-1$ down to 0),
claiming the later slot and decrementing `count[v]`.

`a[p]` then claims the next slot — one to the left.

9. Radix Sort (LSD)

A question to start

Counting sort needs a small key range. What if your keys are five-digit integers — too many distinct values, but each built from tiny digits?

A short history

- **Herman Hollerith**, 1890 US census tabulating machines
- Mechanical card sorters processed one column at a time
- Far older than counting sort itself
- Still used today for fixed-width integer/string keys

The radix sort (LSD) idea

- Start at the **ones** place, run a **stable** counting sort
- Move to **tens**, then **hundreds** — always 10 buckets
- Stability of every pass is not optional — it's required
- Stops once `max_val / place` reaches zero

Radix sort (LSD), step by step

A =

0	1	2	3	4	5	6	7	8	9
8	21	100	329	355	436	457	657	720	839

place = 100

comparisons: 0 (non-comparison sort)

writes: 60

count[] =

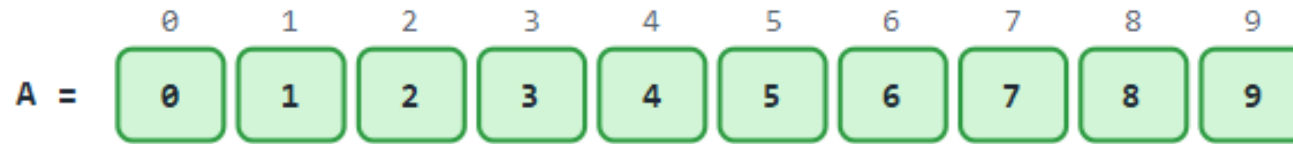
0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

output[] =

0	1	2	3	4	5	6	7	8	9

Done: [8, 21, 100, 329, 355, 436, 457, 657, 720, 839]. Total 60 writes, 0 comparisons. $O(d * (n + 10))$ time, where d is the number of digits; correct because every pass is stable.

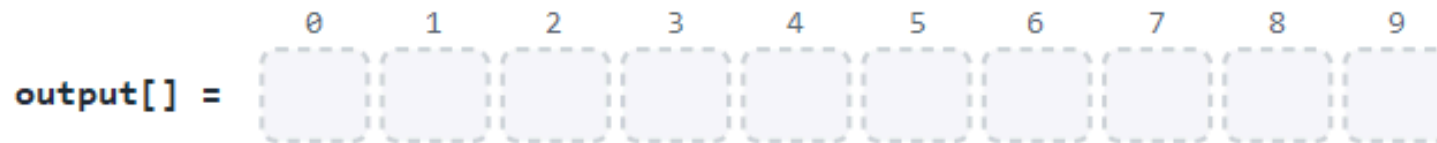
Edge case — single-digit values



place = 1

comparisons: 0 (non-comparison sort)

writes: 20



Done: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]. Total 20 writes, 0 comparisons. $O(d * (n + 10))$ time, where d is the number of digits; correct because every pass is stable.

Code — extracting a digit

```
int get_digit(int x, int place) {  
    return (x / place) % 10;  
}  
  
for (int place = 1;  
     max_val / place > 0;  
     place *= 10) { ... }
```

Code — one digit pass

```
/* stable counting sort,  
   keyed on get_digit(a[i], place) */  
count_and_accumulate(place);  
for (int i = n-1; i >= 0; i--)  
    place_by_digit(a[i], place);  
copy_output_back();
```

Complexity and mistakes

- $O(d \times (n + b))$ — d digits, base $b=10$
- Fixed-width keys: effectively $O(n)$, no log factor
- Common bug: an unstable per-digit pass breaks correctness
- Common bug: sorting from the MOST significant digit first

Mini-quiz

Why does sorting least-significant-digit FIRST work, when running the exact same algorithm most-significant-digit first would not?

Answer

Stability composes upward. Each pass preserves order from all earlier, less-significant digits — MSD-first has no such guarantee to build on.

10. Bucket Sort

A question to start

What if, instead of exact counts, you just threw each value into one of a few buckets by rough magnitude, then cleaned up each small bucket?

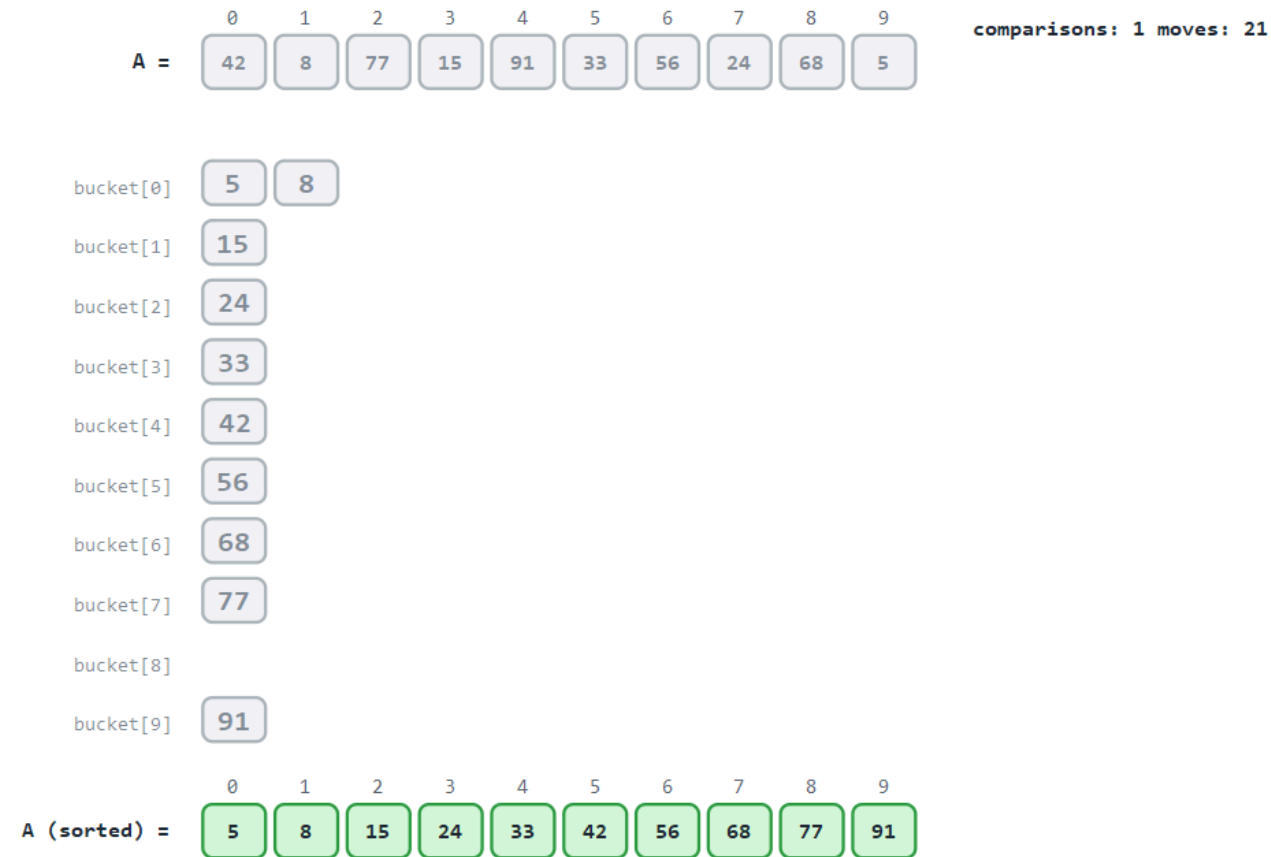
A short history

- The oldest and most general of today's three
- non-comparison sorts — no single inventor or date
- A natural generalization of counting sort
- Depends entirely on the input's actual distribution

The bucket sort idea

- Divide `[0, max_val]` into 10 equal-sized buckets
- Distribute each value into its bucket (tens digit here)
- Sort each small bucket (insertion sort — cheap)
- Concatenate buckets `0..9` — automatically sorted

Bucket sort, step by step



Done: [5, 8, 15, 24, 33, 42, 56, 68, 77, 91]. Total 1 comparisons (from the within-bucket insertion sorts), 21 moves. $O(n)$ average when values are spread evenly; $O(n^2)$ if they all land in one bucket.

Edge case — all in one bucket

A =

0	1	2	3	4	5	6	7	8	9
40	41	42	43	44	45	46	47	48	49

 comparisons: 9 moves: 20

bucket[0]

bucket[1]

bucket[2]

bucket[3]

bucket[4]

40	41	42	43	44	45	46	47	48	49
----	----	----	----	----	----	----	----	----	----

bucket[5]

bucket[6]

bucket[7]

bucket[8]

bucket[9]

A (sorted) =

0	1	2	3	4	5	6	7	8	9
40	41	42	43	44	45	46	47	48	49

Done: [40, 41, 42, 43, 44, 45, 46, 47, 48, 49]. Total 9 comparisons (from the within-bucket insertion sorts), 20 moves. $O(n)$ average when values are spread evenly; $O(n^2)$ if they all land in one bucket.

Code — distribute, then finish

```
int b = (a[i]*BUCKETS)
      / (max_val+1);
bucket[b][bucket_len[b]++]=a[i];
/* ... then insertion-sort each
   bucket, then concatenate */
```

Complexity and mistakes

- **Average $O(n)$** : values spread evenly across buckets
- **Worst $O(n^2)$** : all values collide into one bucket
- Common bug: bucket boundaries that ignore the real data
- **Stable**, if the inner per-bucket sort is stable

Mini-quiz

Bucket sort needs an internal sort per bucket;
counting sort needs nothing similar per value.
What structural difference explains this?

Answer

A bucket covers a **RANGE of values**, so it can hold several genuinely different values still needing relative order — counting sort's counts cannot.

11. Stability, Demonstrated

A question to start

We defined stability in Section 1. Words are not proof. Can we watch the SAME input produce two different results, depending only on the sort?

The demo's setup

- Records are (key, tag) pairs — e.g. 5a, 5b, 5c
- Several records deliberately share a key (a "tie")
- Row 1: **insertion sort** — stable (Section 4)
- Row 2: **selection sort** — unstable (Section 3)

Stability, demonstrated



Done. Unstable sort: [1a, 2a, 2b, 2c, 5b, 5c, 5a, 8b, 8a, 9a]. The keys are sorted identically, but at least one equal-key group can have its tag order differ from the input -- stability is an EXTRA guarantee, not correctness itself.

Edge case — all keys equal



Done. Unstable sort: [6a, 6b, 6c, 6d, 6e, 6f, 6g, 6h, 6i, 6j]. The keys are sorted identically, but at least one equal-key group can have its tag order differ from the input -- stability is an EXTRA guarantee, not correctness itself.

The result, read closely

- Stable (insertion): 5a 5b 5c — exact input order
- Unstable (selection): 5b 5c 5a — 5a pushed to the end
- Not a bug — selection sort's long-range swap in action
- Already-sorted input shows NO difference between them

Which sorts are stable?

Sort	Stable?
Bubble, insertion, merge, counting, radix	Yes
Selection, shell, quick (Lomuto/Hoare)	No
Bucket	Yes, if inner sort is stable

Mini-quiz

Selection sort is unstable. Describe, without code, one general way to force it — or any comparison sort — to become stable.

Answer

Attach the original index to each element,
and break ties in the comparison by that index.
The sort's mechanics never need to change.

12. Comparing the Algorithms, Empirically

A question to start

Big-O hides constant factors. The only way to settle "which sort actually wins on THIS data" is to run them and count. So — let's count.

The setup

- Bubble, selection, insertion, merge, quick (Lomuto)
- All sort the **identical** array, every single time
- Every comparison and write goes through counted helpers
- The tallies are a measurement, not an estimate

Sorting algorithms, compared

bubble	3	6	9	10	15	27	31	38	43	82	comparisons: 45 writes: 52
selection	3	6	9	10	15	27	31	38	43	82	comparisons: 45 writes: 16
insertion	3	6	9	10	15	27	31	38	43	82	comparisons: 33 writes: 35
merge	3	6	9	10	15	27	31	38	43	82	comparisons: 25 writes: 34
quick (Lomuto)	3	6	9	10	15	27	31	38	43	82	comparisons: 25 writes: 34

Ranked by comparison count: 1. merge (25) 2. quick (Lomuto) (25) 3. insertion (33) 4. bubble (45) 5. selection (45). The shape of the input (already sorted, reverse sorted, random) can change which algorithm wins -- there is no single "best" algorithm.

Edge case — already sorted

bubble	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 11 writes: 0
selection	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 66 writes: 0
insertion	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 11 writes: 11
merge	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 20 writes: 44
quick (Lomuto)	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 66 writes: 154

Ranked by comparison count: 1. bubble (11) 2. insertion (11) 3. merge (20) 4. selection (66) 5. quick (Lomuto) (66). The shape of the input (already sorted, reverse sorted, random) can change which algorithm wins -- there is no single "best" algorithm.

Four numbers, one story

Input	Bubble	Quick (Lomuto)
Already sorted (n=12)	11	66
Reverse sorted (n=12)	66	66

No single winner

- Bubble: wins decisively on already-sorted data
- Quick (Lomuto): wins on average for large random data
- Insertion: wins on nearly-sorted data specifically
- The winner always depends on the **shape** of the input

Mini-quiz

On nearly-sorted input (just two values swapped), insertion sort needed only 18 comparisons while selection sort needed the full 66. Why the gap?

Answer

Insertion sort exploits partial order — most elements need one check and stop. Selection sort's full scan ignores how sorted the input is.

13. Choosing a Sorting Algorithm

The decision table

Situation	Best choice
Tiny array, or a recursive sub-array	Insertion sort
Already / nearly sorted	Insertion, or bubble (early exit)
Writes expensive, comparisons cheap	Selection sort

The decision table, continued

Situation	Best choice
Need guaranteed $O(n \log n)$, memory OK	Merge sort
Need average $O(n \log n)$, in place	Quick sort, median-of-3
No recursion allowed	Merge sort (bottom-up), heap sort

The decision table, concluded

Situation	Best choice
Small-integer keys, range $\approx n$	Counting sort
Larger integer/string keys	Radix sort (LSD)
Keys spread evenly over a range	Bucket sort
Ties must keep input order	Any stable sort (Section 11.5)

The real question

- Not "which sort is fastest in general" — there isn't one
- **What do you actually know** about your data?
- **What do you actually need**: a guarantee? stability?
- Section 12 gave you the tool to measure, not guess

Summary

- $O(n^2)$ family: bubble, selection, insertion, shell sort
- $O(n \log n)$ guaranteed: merge sort (top-down, bottom-up)
- $O(n \log n)$ average, in place: quick sort (Lomuto, Hoare)
- Non-comparison, $O(n+k)$: counting, radix, bucket sort

Summary, continued

- Comparison-sorting lower bound: $O(n \log n)$, provably
- Non-comparison sorts beat it by never comparing keys
- Stability: an extra guarantee, not correctness itself
- No single "best" sort — the input's shape decides

Looking ahead

- Week 11: **advanced tree structures**
- Balanced trees maintain sorted order incrementally
- Merge sort's divide-and-conquer reappears, tree-shaped
- Later: **external sorting** — data too big for memory



Questions?

CEN207 Data Structures — Week 10

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027